

Lab 5: Chordy - A Distributed Hash Table

Sevastian Bajenov

October 5, 2026

1 Introduction

In this laboratory work, my task was to implement basic distributed hash table (DHT) using Chord technique. First, I explored simple node setup in a ring and later added a few enhancements such as dynamic node appearance, failure detection and replication. After that, I added a few benchmarks to observe different versions of the distributed hash table and answer important performance questions stated in the laboratory work conditions.

2 Main problems and solutions

The first version, **node1**, only builds the ring. Each node knows its predecessor and successor, and a new node joins by contacting any node already in the ring. Periodic stabilization then moves it to its correct place, and a probe sent around the ring confirms that all nodes are connected in the right order.

The second version, **node2**, turns the ring into a distributed store. Every node keeps a local store and is responsible for the keys between its predecessor and itself. Requests to add or look up a value are forwarded around the ring until they reach the responsible node, which then answers the client. When a new node joins, the node after it hands over the part of its store that now belongs to the newcomer.

The third version, **node3**, adds failure detection. Each node monitors its predecessor and successor and also remembers the node after its successor. When the successor crashes, that node takes its place and the ring is repaired. When the predecessor crashes, the node simply waits for a new one to present itself. The ring survives single crashes, but the data stored on a crashed node is lost.

The fourth version, **node4**, adds replication so that data survives crashes too. Each node keeps a copy of its predecessor's store, and a client receives a confirmation only after the value has been stored by the responsible node and copied to its successor. When a node crashes, its successor takes over the copied data and passes a new copy on to its own successor.

3 Evaluation

All tests were run locally in a single Erlang VM with 14 schedulers. Every ring node and every test client is a separate Erlang process. Nodes stabilize every 1000 ms and the tests wait several stabilization rounds before measuring. Keys are random integers in $[1, 10^9]$.

The first test builds a ring of eight `node1` processes, each joining via a random node that is already in the ring. After stabilization a probe is sent around the ring and the list of visited nodes is compared with the expected order (the nodes sorted by key). Probes sent from two different nodes both visited all eight nodes in exactly the expected order. The ring is therefore correctly connected regardless of which node a newcomer contacts.

In the `node2` setup each client adds its own random keys and then looks all of them up, waiting for every reply before sending the next request. Every configuration was run three times on a fresh ring and the tables show the mean. All lookups succeeded in every run. The node ids follow one of three layouts: *random*; *even*, where every node owns an equal part of the key space; and *uneven*, where one node owns 80% of it. The column *share* is the largest part of the key space owned by a single node.

Clients	Keys per client	Add (ms)	Lookup (ms)	Total (ms)
1	4000	114.4	17.5	132.0
4	1000	116.3	18.2	132.7
1	10000	651.6	86.1	737.8
4	2500	651.4	97.2	744.2

Table 1: One node in the ring

Table 1 answers the first question: four clients adding 1000 elements each take the same time as one client adding 4000. The limiting factor is the single node, which handles one message at a time, so the clients just queue up behind each other. In addition, the store is a list, so every add and lookup scans it and gets slower as the store grows. Adds are about six times slower than lookups because `lists:keystore` copies the list while `lists:keyfind` only reads it.

Layout	Nodes	Share	Add (ms)	Total (ms)
–	1	100%	116.3	132.7
random	2	75.6%	77.0	88.0
random	4	51.6%	41.5	51.3
even	2	50%	45.3	54.6
even	4	25%	22.9	31.2
uneven	2	80%	73.6	85.3
uneven	4	80%	75.7	90.7

Table 2: Adding nodes to the ring (4 clients $\times$ 1000 keys, all clients contact the same node)

Table 2 shows what happens when one and then two more nodes are added. The time follows the largest share almost linearly: with even ids, two nodes halve the time and four nodes divide it by four. With uneven ids, the node owning 80% of the keys does most of the work, so going from two to four nodes gives no improvement at all. Random ids land in between and vary from run to run.

Layout	Nodes	Access	Lookup avg (ms)	Lookup max (ms)	Total (ms)
random	2	same	11.8	13.5	88.0
random	2	spread	15.7	20.4	93.5
random	4	same	10.0	10.3	51.3
random	4	spread	12.4	15.5	49.9
even	2	same	9.8	11.1	54.6
even	2	spread	9.4	10.5	52.6
even	4	same	8.3	8.6	31.2
even	4	spread	7.6	7.8	29.8
uneven	2	same	12.2	13.5	85.3
uneven	2	spread	16.5	21.5	97.5
uneven	4	same	15.1	15.6	90.7
uneven	4	spread	15.7	21.0	92.9

Table 3: All clients contacting the same node vs. each client contacting a different node (4 clients $\times$ 1000 keys)

Table 3 answers whether it matters that all test machines access the same node. Locally it does not change the total time, since a key is always stored by the same node regardless of where the request enters, and forwarding a message costs only microseconds. With even ids the two strategies are identical. With uneven or random ids, spreading the clients increases the maximum lookup time: the clients now finish their adds at different times, so a client that starts its lookups early has them queued behind other clients’ slow adds at the heavily loaded node.

Layout	Access	Share	Add (ms)	Lookup max (ms)	Total (ms)
random	same	46.3%	195.0	36.8	227.1
random	spread	48.6%	207.7	70.8	239.7
even	same	25%	110.2	27.6	136.4
even	spread	25%	116.1	28.1	141.7
uneven	same	80%	449.3	64.0	507.1
uneven	spread	80%	442.7	123.3	502.4

Table 4: Ten thousand elements in a ring of four nodes (4 clients $\times$ 2500 keys)

With ten thousand elements (Table 4 and the last two rows of Table 1), 2.5 times more elements make a single node 5.6 times slower (132 ms vs. 738 ms), because each operation costs time proportional to the size of the store. Distributing the keys therefore pays off even more: four even nodes are 5.4 times faster than one node, while the uneven ring is still only 1.5 times faster.,

In **node3** every node monitors its predecessor and successor and keeps track of its successor’s successor. The test adds 1000 keys, kills a random node, waits three seconds and probes the ring again.

Nodes	Nodes after crash (probe)	Keys found	Keys lost
2	1	511	489
4	3	948	52
6	5	890	110
8	7	934	66

Table 5: One node killed in a ring of **node3** nodes (1000 keys)

In every case the probe found a closed ring with one node fewer, so the predecessor of the dead node used its next node as the new successor and the ring was repaired. The keys stored by the dead node were lost, and the amount lost is simply that node’s share of the key space. The crash of two neighboring nodes at the same time would break the ring, since the next node would be dead as well.

In **node4** each node keeps a replica of its predecessor’s store. A client gets its confirmation from the successor, only after the key has been stored by the owner and replicated. When a node dies its successor merges the replica into its own store and sends the new store to its successor, so the keys are replicated again. To cover the race between a join and an add, a replicate message is accepted only from the current predecessor, otherwise it is passed on to the node that has just joined. The test adds 1000 keys, adds 1000 more while a new node joins, and then kills two nodes one after the other. The same scenario was run with **node3** for comparison.

Nodes	Module	After join	After 1st crash	After 2nd crash
4	node3	2000	1914	1709
4	node4	2000	2000	2000
6	node3	2000	1884	1813
6	node4	2000	2000	2000
8	node3	2000	1944	1577
8	node4	2000	2000	2000

Table 6: Keys found out of 2000 confirmed keys after a join and two crashes

As Table 6 shows, **node4** did not lose a single confirmed key, also after the second crash, which only works because the merged store is replicated again. The solution still has limits: two neighboring nodes failing at the same time lose both the store and its only copy; there is a window of about one stabilization round after a crash in which some keys have only one copy; and a node that is wrongly detected as dead (only slow or temporarily unreachable) would leave two nodes believing they are responsible for the same keys. Inside a single Erlang VM a 'DOWN' message is always correct, but between real machines it only means that the heartbeats stopped, so trusting it is not a reliable solution.

4 Conclusions

In this laboratory work I had the opportunity to familiarize myself with the concept of Chord, a distributed hash table. I also managed to explore the concepts of failure detection and replication in relation to each other. The final version of the system is still vulnerable to certain situations like multiple node corruption or false failure detection. In my opinion, further enhancements of this DHT should focus on fault tolerance as its primary function is storage. With more advanced control mechanisms the performance is going to degrade but it can be compensated by using fingers, for example, from the original Chord paper to increase lookup speed.